Hands-on Exercise — PostgreSQL CRUD from Python

Scenario: Co-working Space Booking System

A co-working company wants to maintain its meeting rooms and room bookings.

Part 1 — Create the database manually

Open pgAdmin / PostgreSQL Query Tool and create:

```
CREATE DATABASE coworking_demo;
```

That is the **only database work to be done directly in PostgreSQL**.

Everything below must be done from Python.

Part 2 — Create the PyCharm project

Create:

```
coworking_db_project
```

Create:

```
main.py
```

Install the PostgreSQL driver if required:

```
python -m pip install psycopg2-binary
```

Connect Python to:

```
Database: coworking_demo
Host: localhost
Port: 5432
User: postgres
Password: your PostgreSQL password
```

First confirm:

```
Connected to PostgreSQL
```

Part 3 — Create tables from Python

Using Python, create these two tables.

`rooms`

Column	Requirement
room_id	Primary Key
room_name	Text
capacity	Integer
hourly_rate	Decimal
status	Text

`bookings`

Column	Requirement
booking_id	Primary Key
room_id	Foreign Key
booked_by	Text
booking_date	Date

Column	Requirement
hours	Integer

Relationship:

```
ROOM
  1
  |
  |
 MANY
BOOKINGS
```

A room can have many bookings.

Part 4 — Insert data

Using Python, insert at least **4 rooms**.

Use your own data.

Example room types could be:

```
Focus Room
Conference Room
Creative Studio
Board Room
```

Then insert at least **6 bookings** for different rooms and dates.

Do not manually insert anything through pgAdmin.

Part 5 — READ

Using Python:

1. Display all rooms.
2. Display all bookings.

3. Show only rooms with capacity greater than `6`.
4. Show only available rooms.
5. Show bookings made by a particular person.

Part 6 — UPDATE

Perform these updates from Python:

1. Change the hourly rate of one room.
2. Change one room's status from:

```
Available
```

to:

```
Maintenance
```

3. Increase the number of booked hours for one booking.

After each update, display the modified record.

Part 7 — DELETE

Delete one booking.

Then display all bookings again and verify that it has disappeared.

Do **not** delete the room connected to other bookings yet.

Part 8 — JOIN

Display booking information together with room information.

The output should contain:

```
Booking ID
Booked By
Room Name
Booking Date
Hours
Hourly Rate
```

Conceptually:

```
bookings
    ↓ room_id
rooms
    ↓
Combined Result
```

Part 9 — Calculate something useful

Using SQL through Python, answer:

Question 1

How many bookings does each room have?

Expected type of result:

```
Focus Room       2
Board Room       1
Creative Studio  3
```

Question 2

Which room has the highest hourly rate?

Question 3

Calculate the booking cost:

```
Booking Cost = Hours × Hourly Rate
```

Display:

```
Booked By | Room | Hours | Cost
```
